```
    arry_to_list=p.tolist()
    l.append(arry_to_list)
show_error(l)
```

The basic code is taken from *https://iamtrask.github.io/2015/07/27/python-network-part2/* and updated for the practical understanding of the learning rate and graph output.

Output

After adding debug prints, we get the output as shown in the box below, for the above program. You can see that after training is complete, the error is almost zero and the actual output is close to the desired output.

```
Inputs:x
[[0 0 1]
 [0 1 1]
 [1 0 1]
 [1 1 1]]

Expected Output: d
[[0]
 [1]
 [1]
 [0]]

Initial weight in Layer1: w1
[[-0.16595599  0.44064899 -0.99977125 -0.39533485]
 [-0.70648822 -0.81532281 -0.62747958 -0.30887855]
 [-0.20646505  0.07763347 -0.16161097  0.370439  ]]

Initial weight in Layer2: w2
[[-0.5910955 ]
 [ 0.75623487]
 [-0.94522481]
 [ 0.34093502]]

Error Before Training:
[[ 0.47372957]
 [-0.51104304]
 [-0.45615914]
 [ 0.54470837]]

Actual Output Before Training: z
[[ 0.47372957]
 [ 0.48895696]
 [ 0.54384086]
 [ 0.54470837]]

*************After Training complete*******************


Weights in Layer1 After training: w1
[[ 4.05751199  3.77592492 -6.07774883 -3.91512755]
 [-1.92447764 -5.52099478 -6.39028245 -3.19429746]
 [ 0.57447316 -1.83960582  2.41541223  5.23881505]]

Weights in Layer2 After training: w2
[[-6.01822566]
 [ 6.39184307]
 [-9.63829175]
 [ 6.90678217]]

Error After Training:
[[ 0.00702213]
 [-0.00899048]
 [-0.00784838]
 [ 0.01047911]]

Actual Output After Training: z
[[ 0.00702213]
 [ 0.99100952]
 [ 0.99215162]
 [ 0.01047911]]
```

If you use the weights that we get after training and the new input (other than those that we used in training), you will get the actual output in terms of trained inputs only.

If you closely look at the code, the main things are happening at Lines 63 and 65. These two lines are just using the derivation equations derived in Box 2.

Learning rate

Please refer to Figures 8, 9, 10 and 11. The graphs given in these figures are plotted for different test scenarios by slightly modifying the code.

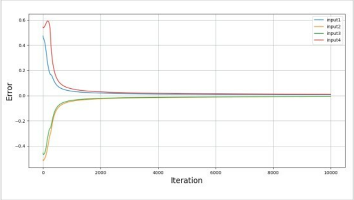

Figure 8: Case 1

Case 1 (Figure 8)

- Keep updating weights by calculating the slope for 10,000 iterations (same as code above).
- We actually don't need these many iterations to reach an error close to zero.

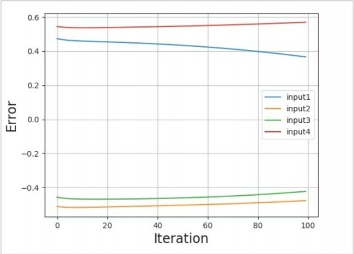

Figure 9: Case 2

Case 2 (Figure 9)

- If the number of iterations is too low, in our case, 100, we will not get an error close to zero. And our actual output will not be close to the desired output. It will be unpredictable.

Case 3 (Figure 10)

- There is a way with which we can reduce the error close to zero for a small number of iterations. The only update you will need is to multiply the slope by some constant.
- Here, I used the constant 20, which is called the learning rate.

```

    w2 -= 20*y.T.dot(l2_delta)        #new_weight(w2)=old_weight(w2)-change in weight(i.e.derivative w.r.t w2)
    w1 -= 20*l0.T.dot(l1_delta)    #new_weight(w1)=old_weight(w1)-change in weight(i.e.derivative w.r.t w1)

```